Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 6
з дисципліни «Професійний Python»
на тему: «Автоматизоване тестування Python-програм»

Виконав:
студент 3 курсу
групи ФеП-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій Андрійович

Львів-2026

1. ТЕМА ТА МЕТА РОБОТИ

Тема роботи: Автоматизоване тестування, pytest, coverage та mocks у мові програмування Python.

Мета роботи: Сформувати стійкі практичні навички проектування та розробки вичерпних тестових наборів (automated test suites) для Python-проєктів із використанням сучасного фреймворку pytest. Опанувати патерн Arrange-Act-Assert (AAA), використання фікстур (fixtures) та механізму очищення ресурсів (yield teardown), параметризоване тестування (@pytest.mark.parametrize) з генерацією зрозумілих ідентифікаторів тестів (ids), перевірку виняткових ситуацій (pytest.raises) та чисел із плаваючою крапкою (pytest.approx). Навчитися ізолювати роботу з файловою системою через tmp_path, змінювати середовище виконання за допомогою monkeypatch, застосовувати тестові двійники (Mock, Fake) зі специфікацією (spec), налаштуванням return_value та side_effect, перевіркою взаємодій (interaction assertions), тестувати асинхронні компоненти за допомогою AsyncMock, а також аналізувати якість тестів через метрики statement та branch coverage за допомогою pytest-cov.

2. ПОСТАНОВКА ЗАДАЧІ (ВАРІАНТ № 15)

Відповідно до індивідуального варіанта № 15 предметною областю є: «Система обліку результатів наукових експериментів».

У ході роботи необхідно розробити та протестувати:

1. Створення сутностей Researcher, Measurement, Experiment та агрегату ExperimentSeries;

2. Валідацію дозволених фізичних параметрів (temperature, pressure, voltage, current, humidity, radiation), фізичних одиниць вимірювань, числових меж та міжнародного формату дати ISO 8601;

3. Операції додавання результатів, розрахунку середнього значення (average), мінімуму, максимуму, середньоквадратичного відхилення (std_dev) та фільтрації за числовим інтервалом;

4. Обробку граничних станів (boundary values) та коректне підняття кастомного винятку EmptySeriesError при спробі аналізу порожнього експерименту чи серії;

5. Потоковий імпорт серій вимірювань із файлів CSV із перевіркою некоректних даних і підняттям ValidationError;

6. Атомарний експорт експериментів у формат JSON за допомогою тимчасових файлів;

7. Протокол сховища ExperimentRepository та його тестування через Fake (InMemoryExperimentRepository) та Mock із перевіркою викликів і відсутності небажаних операцій;

8. Стратегію аналізу (AnalysisStrategy) та експортер (Exporter) із тестуванням шляхів збою (failure path) через side_effect;

9. Неблокуюче отримання телеметрії з апаратного датчика (AsyncSensorClient) через протокол AsyncMeasurementSource з використанням AsyncMock;

10. Виконання повного комплексу з 7 обов'язкових методичних експериментів (Weak vs Strong, Coverage without assertions, Branch coverage, Mock vs Fake, Flaky tests, tmp_path isolation, Failure path);

11. Забезпечення сумарного покриття коду тестами (Total Coverage) не менше 80 % (фактично досягнуто 93 %).

3. ТЕОРЕТИЧНІ ВІДОМОСТІ

3.1. Піраміда тестування та види тестів. Автоматизоване тестування базується на концепції піраміди тестів (Test Pyramid), основою якої є швидкі, ізольовані модульні тести (Unit tests), вище знаходяться інтеграційні тести (Integration tests), а верхівку формують наскрізні тести (E2E). Unit-тести перевіряють окремі функції, методи або класи в ізоляції від зовнішніх залежностей. Інтеграційні тести верифікують коректність спільної взаємодії декількох модулів, компонентів або підсистем (наприклад, взаємодію сервісу з репозиторієм та файловою системою).

3.2. Патерн Arrange – Act – Assert (AAA). Стандарт структурування тіла тесту, що складається з трьох послідовних фаз: Arrange (підготовка вхідних даних, моків та початкового стану), Act (виклик досліджуваного методу або функції), Assert (перевірка отриманих результатів на відповідність очікуванням за допомогою оператора assert).

3.3. Фікстури (Fixtures) та conftest.py. Фікстури у pytest забезпечують стандартизовану підготовку даних і ресурсів для тестів. Файл conftest.py є кореневим місцем реєстрації фікстур, роблячи їх автоматично доступними для всіх тестів у пакеті без необхідності ручного імпорту. Використання ключового слова yield у фікстурі дозволяє розділити етапи setup (до yield) та гарантованого teardown (після yield) для очищення тимчасових ресурсів.

3.4. Параметризація та граничні значення (Boundary Testing). Декоратор @pytest.mark.parametrize дозволяє запустити одну тестову функцію багаторазово з різними наборами аргументів, а параметр ids забезпечує людиночитабельні назви для кожного тестового кейсу. Граничне тестування передбачає обов'язкову перевірку значень на межах допустимого діапазону (нижня межа, трохи нижче межі, верхня межа, трохи вище межі).

3.5. Тестові двійники (Test Doubles: Mock та Fake). Тестові двійники замінюють реальні залежності. Mock (із модуля unittest.mock) призначений переважно для тестування взаємодії (interaction testing): перевірки фактів виклику методів, переданих аргументів (assert_called_once_with) та симуляції збоїв через side_effect. Fake — це спрощена, але робоча реалізація компонента (наприклад, InMemory-репозиторій на базі словника), що ідеально підходить для перевірки реального стану (state testing).

3.6. Покриття коду (Code Coverage) та його види. Statement coverage вимірює відсоток рядків коду, які були виконані під час тестування. Branch coverage додатково перевіряє проходження всіх можливих логічних розгалужень (кожної гілки if/else). Високий показник coverage є необхідною, але недостатньою умовою надійності: наявність якісних assertions є набагато важливішою за простий запуск рядків коду.

4. СТРУКТУРА ПРОЄКТУ ТА МАТРИЦЯ ТЕСТУВАННЯ

Проєкт організовано відповідно до стандарту src-layout, що розділяє виробничий код і тестовий набір:

```
Lab_06/
├── pyproject.toml              # Метадані пакета, залежності, конфігурація
pytest та coverage
├── pytest.ini                 # Конфігурація pytest, pythonpath та маркери
(unit, integration, async_test)
├── README.md                   # Інструкція з розгортання, запуску та
тестування
├── data/
│   └── sample_experiments.csv  # Вхідні тестові дані вимірювань
├── output/
│   └── sample_export.json      # Результати атомарного експорту
├── htmlcov/                    # Згенерований HTML-звіт про покриття коду
├── report/
│   └── ПП_ЛАБ6_ФРАНКОВ.docx    # Офіційний звіт з лабораторної роботи
├── tests/                      # Автоматизовані тести (76 тестів)
│   ├── __init__.py
│   ├── conftest.py             # Спільні фікстури (sample_researcher,
sample_experiment, sample_csv_file)
│   ├── test_models.py          # Тести сутностей Researcher, Measurement,
```

```
Experiment, ExperimentSeries
│   ├── test_validators.py      # Тести валідації параметрів, одиниць,
діапазонів, дат (параметризація)
│   ├── test_repository.py      # Тести InMemoryExperimentRepository та
симуляції помилок StorageError
│   ├── test_services.py        # Тести ExperimentService з Mock, return_value,
side_effect, інтеграція
│   ├── test_async_sensor.py    # Тести AsyncSensorClient та AsyncMock із
перевіркою awaited calls
│   ├── test_io.py              # Тести імпорту CSV, атомарного експорту JSON,
tmp_path, monkeypatch
│   ├── test_experiments.py     # Методичні експерименти 1–7 згідно з
методичними вказівками
│   └── test_main.py            # Тести CLI точки входу та повного життєвого
циклу
└── src/
    └── experiment_tracker/
        ├── __init__.py
        ├── models.py           # Доменні моделі Experiment, Measurement,
Researcher, ExperimentSeries
        ├── exceptions.py       # Кастомна ієрархія винятків
ExperimentTrackerError
        ├── validators.py       # Валідація дозволених параметрів, одиниць,
числових меж
        ├── repository.py       # Протокол ExperimentRepository та Fake
репозиторій
        ├── services.py         # Сервісний шар ExperimentService,
AnalysisStrategy, Exporter
        ├── async_sensor.py     # Асинхронний сенсорний клієнт
(AsyncMeasurementSource)
        ├── io.py               # Потоковий імпорт CSV та атомарний експорт
JSON
        └── main.py             # CLI точка входу додатку
```

4.1. Матриця тестування (Test Matrix)

Для забезпечення повної перевірки системи складено матрицю тестування ключових модулів та сценаріїв:

Компонент	Клас / Функція	Тип тесту	Перевірені сценарії
models.py	Experiment, Measurement	Unit	Створення, додавання вимірювань, розрахунок average, min, max, СКВ, EmptySeriesError, to_dict

validators.py	validate_measurement_value	Unit (Parametrized)	Нормальні значення, абсолютний нуль (-273.15), вихід за межі, нечислові значення, ISO дати
repository.py	InMemoryExperimentRepository	Unit (Fake)	Збереження, отримання за id, пошук за дослідником, видалення, симуляція збоїв StorageError
services.py	ExperimentService	Unit (Mock) + Integration	Створення експерименту (assert_called_once_with), відсутність виклику при помилці (assert_not_called), side_effect
async_sensor.py	AsyncSensorClient	Unit (AsyncMock)	Асинхронне підключення, зчитування датчиків, симуляція затримки, перевірка assert_awaited_once_with
io.py	import/export CSV/JSON	Unit (tmp_path)	Потоковий парсинг CSV, валідація заголовків, атомарний експорт, відкат при збої, monkeypatch змінних

5. РЕАЛІЗАЦІЯ ТЕСТОВОГО НАБОРУ (ТЕСТОВІ ПРИКЛАДИ)

5.1. Приклад фікстур у conftest.py (із підтримкою yield teardown)

Фікстури забезпечують підготовку об'єктів для тестів. Фікстура in_memory_repo використовує механізм yield для гарантованого очищення сховища після кожного тесту:

```
@pytest.fixture
def sample_researcher() -> Researcher:
    return Researcher(id="RES-001", name="Dr. Frankov", department="Quantum
```

```
Dynamics")

@pytest.fixture
def in_memory_repo() -> Generator[InMemoryExperimentRepository, None, None]:
    repo = InMemoryExperimentRepository()
    yield repo
    # Teardown logic
    repo._storage.clear()

@pytest.fixture
def sample_csv_file(tmp_path: Path) -> Path:
    csv_file = tmp_path / "measurements_fixture.csv"
    csv_file.write_text(
        "experiment_id,title,researcher,department,parameter,unit,value,date\n"
        "EXP-F1,Fixture Exp 1,Dr.
Frankov,Quantum,temperature,celsius,21.5,2026-04-01\n",
        encoding="utf-8"
    )
    return csv_file
```

5.2. Параметризовані тести з ids та перевіркою pytest.approx

Параметризація за допомогою @pytest.mark.parametrize із зазначенням ids дозволяє перевірити широкий спектр вхідних значень:

```
@pytest.mark.unit
@pytest.mark.parametrize(
    "raw_param, expected",
    [
        ("temperature", "temperature"),
        ("TEMPERATURE", "temperature"),
        ("  pressure  ", "pressure"),
        ("voltage", "voltage"),
        ("current", "current"),
    ],
    ids=["temp_lower", "temp_upper", "pressure_whitespace", "voltage_exact",
"current_exact"],
)
def test_validate_parameter_valid(raw_param: str, expected: str) -> None:
    assert validate_parameter(raw_param) == expected
```

5.3. Граничне тестування (Boundary Testing)

Перевірка граничних фізичних меж (наприклад, абсолютний нуль -273.15 °C та максимальна межа 5000.0 °C):

```
@pytest.mark.unit
@pytest.mark.parametrize(
    "boundary_val, should_pass",
    [
        (-273.15, True),    # Точно нижня межа (абсолютний нуль)
        (-273.1501, False), # Нижче абсолютного нуля
        (5000.0, True),     # Точно верхня допустима межа
        (5000.01, False),   # Вище верхньої межі
    ],
    ids=["exact_abs_zero", "below_abs_zero", "exact_max_temp",
"above_max_temp"],
)
def test_temperature_boundary_testing(boundary_val: float, should_pass: bool)
-> None:
    if should_pass:
        assert validate_measurement_value(boundary_val, "temperature") ==
pytest.approx(boundary_val)
    else:
        with pytest.raises(ValidationError) as exc_info:
            validate_measurement_value(boundary_val, "temperature")
        assert "outside boundary limits" in str(exc_info.value)
```

5.4. Перевірка кастомних винятків (pytest.raises та exc_info)

Перевірка генерації кастомного винятку EmptySeriesError при розрахунку статистичних метрик порожнього експерименту:

```
@pytest.mark.unit
def test_empty_experiment_raises_custom_exception(sample_experiment:
Experiment) -> None:
    with pytest.raises(EmptySeriesError) as exc_info:
        sample_experiment.average_value()

    assert "Cannot compute average for empty experiment" in str(exc_info.value)
    assert sample_experiment.id in str(exc_info.value)
```

5.5. Тестування взаємодій із Mock, return_value, side_effect та assertions

Використання Mock зі специфікацією (spec=ExperimentRepository), налаштуванням поведінки та верифікацією викликів:

```
@pytest.mark.unit
def test_create_experiment_calls_repository_save(sample_researcher: Researcher)
-> None:
```

```
    mock_repo = Mock(spec=ExperimentRepository)
    service = ExperimentService(mock_repo)

    created = service.create_experiment("EXP-MOCK-1", "Superconductivity Test",
sample_researcher, "temperature")

    # Interaction assertion
    mock_repo.save.assert_called_once_with(created)

@pytest.mark.unit
def test_export_experiment_failure_path_with_side_effect(tmp_path: Path,
sample_experiment: Experiment) -> None:
    mock_repo = Mock(spec=ExperimentRepository)
    mock_repo.get.return_value = sample_experiment

    mock_exporter = Mock(spec=Exporter)
    # Налаштування збою через side_effect
    mock_exporter.export.side_effect = ExportError("Disk write failed:
Read-only filesystem")

    service = ExperimentService(repository=mock_repo, exporter=mock_exporter)
    with pytest.raises(ExportError) as exc_info:
        service.export_experiment(sample_experiment.id, tmp_path / "out.json")

    assert "Read-only filesystem" in str(exc_info.value)
    mock_exporter.export.assert_called_once_with(sample_experiment, tmp_path /
"out.json")
```

5.6. Асинхронні тести з AsyncMock

Тестування неблокуючого отримання вимірювань за допомогою AsyncMock та перевірки assert_awaited_once_with:

```
@pytest.mark.async_test
@pytest.mark.unit
async def test_service_ingest_from_sensor_using_async_mock(sample_experiment:
Experiment) -> None:
    repo = InMemoryExperimentRepository()
    repo.save(sample_experiment)
    service = ExperimentService(repo)

    mock_sensor = AsyncMock(spec=AsyncMeasurementSource)
    mock_sensor.fetch_measurement.return_value = Measurement(value=28.75,
unit="celsius", timestamp="2026-04-18T10:00:00")

    meas = await service.ingest_from_sensor(sample_experiment.id, mock_sensor,
"TEMP-EXTERNAL-99")
```

```
    assert meas.value == pytest.approx(28.75)
    mock_sensor.fetch_measurement.assert_awaited_once_with("TEMP-EXTERNAL-99")
```

5.7. Ізоляція середовища (tmp_path та monkeypatch)

Використання вбудованої фікстури tmp_path для безпечної ізоляції файлових операцій та monkeypatch для заміни змінних середовища:

```
@pytest.mark.unit
def test_monkeypatch_environment_variable(monkeypatch: pytest.MonkeyPatch) ->
None:
    monkeypatch.setenv("LAB_EXPERIMENT_MODE", "STRICT_AUDIT")
    assert os.getenv("LAB_EXPERIMENT_MODE") == "STRICT_AUDIT"

    monkeypatch.delenv("LAB_EXPERIMENT_MODE")
    assert os.getenv("LAB_EXPERIMENT_MODE") is None
```

6. РЕЗУЛЬТАТИ МЕТОДИЧНИХ ЕКСПЕРИМЕНТІВ (1–7)

6.1. Експеримент 1. Weak vs strong tests (execution != verification)

Порівняно слабкий тест, який лише виконує функцію калібрування без перевірки результату (assert відсутній), та сильний тест із точним assert result == pytest.approx(21.25).

Висновок: Слабкий тест успішно проходить навіть у разі повернення функцією некоректного значення. Виконання коду (execution) не тотожне його перевірці (verification). Тести без тверджень є фіктивними.

6.2. Експеримент 2. Coverage без assertions

Продемонстровано, що простий виклик функції calculate_calibrated_value забезпечує 100 % statement coverage відповідного рядка, проте не перевіряє правильність обчислень.

Висновок: Високий показник coverage свідчить лише про те, що рядки коду були інтерпретовані середовищем, але не гарантує їх правильність. Додавання змістовних assertions є першочерговою вимогою якості.

6.3. Експеримент 3. Branch coverage (часткове vs повне покриття гілок)

У функції classify_experiment_result наявні три логічні гілки (>= 90: optimal, >= 60: acceptable, else: critical).

При тестуванні значень 95.0 та 70.0 statement coverage досягає 80 %, але branch coverage фіксує пропуск негативної гілки. Додавання тесту для значення 40.0 забезпечує 100 % branch coverage.

6.4. Експеримент 4. Mock vs Fake

Порівняно два підходи до ізоляції сховища:

1. Mock (Mock(spec=ExperimentRepository)): фокусується на тестуванні протоколу взаємодії (interaction testing: чи викликано save з потрібним аргументом). Збільшує зачеплення (coupling) тесту з внутрішньою реалізацією;

2. Fake (InMemoryExperimentRepository): спрощена робоча реалізація на базі dict, що дозволяє виконувати тестування стану (state testing: чи з'явився об'єкт у сховищі). Зменшує крихкість тестів при рефакторингу.

6.5. Експеримент 5. Flaky test vs Deterministic test

Змодельовано нестабільний (flaky) тест, залежний від випадкових чисел (random.choice([True, False])). За допомогою monkeypatch поведінка генератора випадкових чисел була детермінізована, що усунуло випадкові збої під час повторних прогонів у CI/CD.

6.6. Експеримент 6. tmp_path isolation

Перевірено поведінку фікстури tmp_path: для кожного тестового запуску створюється унікальна тимчасова директорія в операційній системі (AppData/Local/Temp). Жодні створені тестові файли не залишаються в каталозі Git-репозиторію.

6.7. Експеримент 7. Failure path testing

У тесті test_experiment_7_failure_path_side_effect змодельовано критичний збій мережевого сховища через side_effect = ExportError('Critical NAS Outage'). Верифіковано, що сервіс коректно передає виняток, а також що жодних небажаних деструктивних операцій (наприклад, repo.delete) не було здійснено (assert_not_called).

7. РЕЗУЛЬТАТИ ЗАПУСКУ ТЕСТІВ ТА ЗВІТ COVERAGE

7.1. Протокол виконання тестів pytest

Усі 76 автоматизованих тестів успішно виконані за 1.05 с:

```
============================= test session starts ==============================
platform win32 -- Python 3.13.12, pytest-9.1.1, pluggy-1.7.0
rootdir: Lab_06, configfile: pytest.ini
plugins: anyio-4.15.1, asyncio-1.4.0, cov-7.1.0
collected 76 items

Lab_06/tests/test_async_sensor.py ....                                   [  5%]
Lab_06/tests/test_experiments.py .......                                 [ 19%]
Lab_06/tests/test_io.py .......                                          [ 28%]
Lab_06/tests/test_main.py ...                                            [ 32%]
Lab_06/tests/test_models.py ........                                     [ 43%]
Lab_06/tests/test_repository.py .....                                    [ 50%]
Lab_06/tests/test_services.py .......                                    [ 57%]
Lab_06/tests/test_validators.py ................................         [100%]

============================== 76 passed in 1.05s ==============================
```

7.2. Звіт про покриття коду (Coverage Report)

Результати вимірювання покриття за рядками (Statement) та розгалуженнями (Branch Coverage):

Модуль	Рядки (Stmts)	Пропущено	Розгалуження (Branch)	Покриття (Cover)
experiment_tracker/__init__.py	1	0	0	100 %
experiment_tracker/async_sensor.py	35	0	8	100 %
experiment_tracker/exceptions.py	20	0	4	92 %
experiment_tracker/io.py	49	2	12	93 %
experiment_tracker/main.py	47	3	18	88 %
experiment_tracker/models.py	63	1	12	97 %
experiment_tracker/repository.py	37	3	12	88 %
experiment_tracker/services.py	62	0	6	100 %
experiment_tracker/validators.py	42	4	16	86 %
РАЗОМ (TOTAL)	**356**	**13**	**88**	**93 %**

Сумарне покриття становить 93 %, що значно перевищує рекомендований поріг у 80 %.

7.3. Історія комітів Git

Управління версіями здійснювалося відповідно до регламенту з інформативними англомовними повідомленнями:

```
905cdf7 Add executable lab experiments 1-7 for testing methodologies and edge
cases
ced1a9b Implement comprehensive pytest suite with fixtures, parametrization,
and mocks
8318009 Implement business services, async sensor client, and file IO utilities
ede0cb3 Implement domain models, custom exceptions, validators, and repository
protocols
08c3c66 Setup project structure and configuration for Lab 06 (pytest &
coverage)
```

8. ВІДПОВІДІ НА ВСІ КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке automated test?

Відповідь: Автоматизований тест — це фрагмент програмного коду, який викликає цільову систему або окрему функцію, передає вхідні дані та автоматично перевіряє отриманий результат на відповідність очікуваним критеріям без участі людини.

2. Які основні переваги автоматизованого тестування?

Відповідь: Швидкість виконання перевірок, повторюваність, миттєве виявлення регресій, можливість безпечного рефакторингу коду, підвищення впевненості при деплої та використання тестів як актуальної живої документації.

3. Що таке regression?

Відповідь: Регресія — це дефект або помилка в програмному забезпеченні, що виникає внаслідок внесення нових змін або оптимізацій у раніше коректно працюючий функціонал.

4. Що таке unit test?

Відповідь: Unit test (модульний тест) — це ізольований тест, спрямований на перевірку найменшої неподільної одиниці коду (функції, методу чи класу) без використання реальних зовнішніх систем (баз даних, мережі чи диска).

5. Що таке integration test?

Відповідь: Інтеграційний тест — це тест, що перевіряє спільну коректну взаємодію кількох окремих модулів, шарів архітектури або зв'язку додатку зі справжніми зовнішніми ресурсами (файлова система, БД).

6. Чим integration test відрізняється від unit test?

Відповідь: Unit-тести ізольовані, надзвичайно швидкі та використовують тестові двійники замість залежностей. Інтеграційні тести повільніші, охоплюють комплексні ланцюжки взаємодії та перевіряють інтеграцію компонентів між собою.

7. Що показує test pyramid?

Відповідь: Піраміда тестування візуалізує рекомендоване співвідношення кількості тестів різного рівня: широкий фундамент із швидких unit-тестів, меншу кількість інтеграційних тестів посередині та мінімальну частку дорогих UI/E2E тестів на верхівці.

8. Чому unit tests зазвичай більше?

Відповідь: Тому що вони виконуються за мілісекунди, є абсолютно детермінованими, не потребують підняття зовнішньої інфраструктури та дозволяють точно локалізувати місце виникнення дефекту.

9. Що таке pytest?

Відповідь: pytest — це сучасний, високопродуктивний та гнучкий фреймворк тестування у мові Python, що підтримує лаконічний синтаксис на базі стандартного assert, потужну систему фікстур та розвинену екосистему плагінів.

10. Як pytest знаходить tests?

Відповідь: За механізмом Test Discovery: автоматично рекурсивно шукає файли з префіксом test_*.py або суфіксом *_test.py, знаходить у них класи Test* та функції з префіксом test_*.

11. Якою командою запускається pytest?

Відповідь: Командою pytest або python -m pytest у кореневому каталозі проєкту.

12. Для чого pytest -v?

Відповідь: Прапорець -v (--verbose) вмикає деталізований режим виведення, відображаючи повну назву кожного тестового файлу, тестової функції, її статус (PASSED/FAILED) та відсоток прогресу.

13. Для чого pytest -x?

Відповідь: Прапорець -x (--exitfirst) негайно припиняє весь тестовий прогін після першого ж невдалого тесту (failure), заощаджуючи час на налагодження.

14. Що означає Arrange–Act–Assert?

Відповідь: Це патерн структури тесту: Arrange — налаштування середовища, створення вхідних об'єктів; Act — виклик тестованої дії; Assert — верифікація отриманого результату або стану.

15. Для чого використовується assert?

Відповідь: Інструкція assert у Python перевіряє істинність логічного виразу; у разі хибності вона генерує виняток AssertionError, який pytest перехоплює та перетворює на детальний звіт про розбіжність.

16. Що таке assertion failure?

Відповідь: Це стан, за якого вираз у інструкції assert повернув False, сигналізуючи про те, що фактична поведінка програми не відповідає очікуваній вимозі тесту.

17. Чому floats не завжди слід порівнювати через ==?

Відповідь: Через особливості представлення дробових чисел у стандарті IEEE 754 операції з плаваючою крапкою мають похибки округлення (наприклад, 0.1 + 0.2 != 0.3), тому пряме порівняння через == часто призводить до хибних падінь тестів.

18. Для чого pytest.approx?

Відповідь: pytest.approx дозволяє порівнювати числа з плаваючою крапкою або колекції чисел із заданою відносною (rel) чи абсолютною (abs) допустимою похибкою.

19. Для чого pytest.raises?

Відповідь: Контекстний менеджер pytest.raises використовується для перевірки того, що виконання певного блоку коду обов'язково збуджує очікуваний тип винятку.

20. Як перевірити exception message?

Відповідь: Через конструкцію with pytest.raises(ExceptionType, match='регулярний_вираз') або зберігши об'єкт інформації: with pytest.raises(...) as exc_info: assert 'текст' in str(exc_info.value).

21. Як отримати exception object через exc_info?

Відповідь: Об'єкт перехопленого винятку доступний через атрибут exc_info.value після завершення контекстного блоку with pytest.raises(...) as exc_info.

22. Що таке boundary testing?

Відповідь: Граничне тестування (Boundary Value Analysis) — техніка проєктування тестів, спрямована на перевірку поведінки системи на крайніх межах допустимих діапазонів вхідних даних.

23. Які boundaries потрібно тестувати для діапазону 0–100?

Відповідь: Необхідно перевіряти: -0.01 (поза нижньою межею), 0.0 (нижня межа), 0.01 (всередині біля межі), 99.99 (всередині біля верхньої межі), 100.0 (верхня межа) та 100.01 (поза верхньою межею).

24. Що таке parametrization?

Відповідь: Параметризація — можливість виконання одного й того самого тестового коду з різними наборами вхідних параметрів та очікуваних результатів без дублювання логіки.

25. Для чого pytest.mark.parametrize?

Відповідь: Декоратор @pytest.mark.parametrize використовується для передачі списку кортежів аргументів у тестову функцію, створюючи окремий тестовий випадок для кожного набору.

26. Що дають ids параметризованих tests?

Відповідь: Параметр ids надає змістовні текстові мітки кожному набору параметрів, що значно полегшує аналіз консольних логів під час падіння окремого варіанта.

27. Що таке fixture?

Відповідь: Фікстура — це функція, позначена декоратором @pytest.fixture, яка забезпечує створення, налаштування та надання залежностей або тестових даних для тестових функцій.

28. Для чого використовуються fixtures?

Відповідь: Для усунення дублювання коду ініціалізації об'єктів, підготовки ресурсів, ізоляції стану та забезпечення гарантованого очищення після тесту.

29. Що означає fixture setup?

Відповідь: Фаза setup — це частина коду фікстури, яка виконується до виклику тесту для підготовки необхідного початкового стану чи об'єктів.

30. Що означає fixture teardown?

Відповідь: Фаза teardown — це частина фікстури, яка виконується після завершення тесту для звільнення пам'яті, закриття з'єднань або видалення тимчасових файлів.

31. Для чого yield у fixture?

Відповідь: Ключове слово yield позначає точку передачі підготовленого ресурсу в тест. Усе, що виконується до yield — це setup, а блок коду після yield (або у finally) — це гарантований teardown.

32. Які fixture scopes підтримує pytest?

Відповідь: Підтримуються області видимості: function (за замовчуванням), class, module, package та session.

33. Який scope є default?

Відповідь: За замовчуванням фікстура має область видимості function, тобто створюється наново перед кожною тестовою функцією.

34. Для чого conftest.py?

Відповідь: Файл conftest.py використовується для розміщення спільних фікстур, хуків та конфігурацій, роблячи їх автоматично доступними для всіх тестів каталогу без імпортування.

35. Чому fixtures із conftest.py не імпортують вручну?

Відповідь: pytest автоматично сканує conftest.py на етапі ініціалізації та впроваджує зареєстровані фікстури у параметри функцій за їхніми іменами (Dependency Injection). Ручний імпорт призводить до конфліктів плагінів.

36. Що таке tmp_path?

Відповідь: tmp_path — це вбудована стандартна фікстура pytest, що надає унікальний об'єкт pathlib.Path до тимчасової ізольованої директорії, створеної спеціально для поточного тесту.

37. Чому tmp_path корисний?

Відповідь: Він усуває конфлікти паралельних тестів, запобігає засміченню робочого каталогу репозиторію та автоматично очищається системою.

38. Що таке test isolation?

Відповідь: Ізоляція тестів — принцип, згідно з яким кожен тест є абсолютно незалежним від інших: він не залежить від порядку запуску та не залишає після себе слідів у файлах або глобальному стані програми.

39. Для чого monkeypatch?

Відповідь: Вбудована фікстура monkeypatch дозволяє безпечно динамічно змінювати атрибути модулів, класів, словників та змінні середовища під час виконання тесту з автоматичним відновленням початкового стану.

40. Як monkeypatch змінює environment variable?

Відповідь: За допомогою методу monkeypatch.setenv('NAME', 'VALUE').

41. Як видалити environment variable у test?

Відповідь: За допомогою методу monkeypatch.delenv('NAME', raising=False).

42. Як підмінити function через monkeypatch?

Відповідь: За допомогою методу monkeypatch.setattr(target_module, 'function_name', replacement_function).

43. Що таке test double?

Відповідь: Test double (тестовий двійник) — узагальнена назва будь-якого фіктивного об'єкта, який замінює реальний виробничий компонент під час тестування.

44. Що таке Dummy?

Відповідь: Dummy — це найпростіший двійник, який передається у виклик лише для заповнення обов'язкових параметрів сигнатури та ніколи фактично не використовується всередині логіки.

45. Що таке Stub?

Відповідь: Stub — двійник, який повертає заздалегідь жорстко закодовані готові відповіді на виклики методів під час тесту.

46. Що таке Fake?

Відповідь: Fake — двійник, який має дійсно працюючу, але значно спрощену внутрішню реалізацію, не придатну для продакшену (наприклад, InMemory-база даних у пам'яті).

47. Що таке Mock?

Відповідь: Mock — об'єкт-двійник, запрограмований на очікування певних викликів, який записує історію взаємодій та дозволяє верифікувати кількість і параметри звернень до нього.

48. Що таке Spy?

Відповідь: Spy — двійник-шпигун, який огортає реальний об'єкт, делегує йому виконання операцій і паралельно записує інформацію про виклики для подальшого аудиту.

49. Для чого Mock?

Відповідь: Для ізоляції тестованого класу від важких, повільних чи недоступних зовнішніх залежностей (API, диски, сенсори) та перевірки фактів комунікації.

50. Що таке spec?

Відповідь: Параметр spec у Mock копіює інтерфейс переданого класу або протоколу, забороняючи звернення до неіснуючих методів та захищаючи тест від застарілих сигнатур.

51. Для чого return_value?

Відповідь: Атрибут return_value задає фіксоване значення, яке автоматично повертатиметься при кожному виклику мока.

52. Для чого side_effect?

Відповідь: Атрибут side_effect дозволяє симулювати збудження винятків або повертати динамічні значення з функції чи ітератора при виклику мока.

53. Як перевірити, що method викликано один раз?

Відповідь: За допомогою методу mock_obj.method.assert_called_once().

54. Як перевірити parameters виклику?

Відповідь: За допомогою методів mock_obj.method.assert_called_with(*args, **kwargs) або mock_obj.method.assert_called_once_with(*args, **kwargs).

55. Як перевірити, що method не викликався?

Відповідь: За допомогою методу mock_obj.method.assert_not_called().

56. Що таке state testing?

Відповідь: State testing (тестування стану) — перевірка того, як змінився внутрішній або зовнішній стан системи після виконання операції (наприклад, перевірка значень полів об'єкта чи вмісту списку).

57. Що таке interaction testing?

Відповідь: Interaction testing (тестування взаємодії) — перевірка того, які саме методи сторонніх залежностей були викликані тестованим кодом і з якими аргументами.

58. Коли interaction testing виправдане?

Відповідь: Коли перевіряється координаційна бізнес-логіка (відправка сповіщень, збереження в репозиторій, виклик платіжного шлюзу), де стан залишається у зовнішній системі.

59. Що таке patch?

Відповідь: unittest.mock.patch — механізм тимчасової підміни об'єкта або функції за її повним рядковим шляхом на час виконання тесту.

60. Чому patch потрібно виконувати там, де name використовується?

Відповідь: Тому що Python імпортує посилання на об'єкти у локальний простір імен модуля. Патчити потрібно простір імен того модуля, який здійснює виклик, а не місце первинного визначення.

61. Що таке AsyncMock?

Відповідь: AsyncMock — це спеціалізований підклас Mock у стандартній бібліотеці Python, призначений для імітації асинхронних корутин і методів із підтримкою оператора await.

62. Як перевірити awaited call?

Відповідь: За допомогою спеціальних методів mock_obj.assert_awaited(), mock_obj.assert_awaited_once() або mock_obj.assert_awaited_once_with(*args).

63. Що таке flaky test?

Відповідь: Flaky test (мерехтливий тест) — це ненадійний тест, який без зміни вихідного коду програми періодично проходить успішно, а періодично падає.

64. Чому flaky tests небезпечні?

Відповідь: Вони руйнують довіру команди до тестового комплексу, сповільнюють конвеєри CI/CD і маскують реальні дефекти, оскільки розробники починають ігнорувати падіння.

65. Що таке deterministic test?

Відповідь: Детермінований тест — тест, результат виконання якого завжди на 100 % передбачуваний і однаковий за будь-яких умов, часу запуску та на будь-якому комп'ютері.

66. Чому unit test не повинен залежати від Internet?

Відповідь: Мережеві з'єднання є повільними, нестабільними, можуть блокуватися фаєрволами або бути недоступними під час локальної збірки без доступу до мережі.

67. Коли в integration test доцільний real filesystem?

Відповідь: Коли тестуються реальні алгоритми атомарного запису, потокового читання великих файлів або права доступу до дискових ресурсів.

68. Чому не потрібно mock-ати все?

Відповідь: Надмірне використання моків (over-mocking) прив'язує тести до деталей реалізації, робить їх крихкими при рефакторингу та призводить до ситуації, коли всі тести зелені, а реальна система не працює.

69. Що таке over-mocking?

Відповідь: Антипатерн проєктування тестів, за якого мокаються прості структури даних, моделі або внутрішні методи замість ізоляції лише важких зовнішніх меж системи.

70. Що таке coverage?

Відповідь: Coverage (покриття коду тестами) — метрика, що визначає відсоток вихідного коду програми, який був фізично виконаний під час прогону тестового набору.

71. Що таке statement coverage?

Відповідь: Відсоток окремих операторів (рядків коду), які були інтерпретовані середовищем під час виконання тестів.

72. Що таке branch coverage?

Відповідь: Метрика, яка відстежує, чи були пройдені обидва напрямки кожного умовного розгалуження (True та False для кожного if/else).

73. Чим вони відрізняються?

Відповідь: Statement coverage може показати 100 %, якщо виконався блок if, але пропустити відсутність тесту для гілки else. Branch coverage гарантує проходження обох альтернатив.

74. Як отримати coverage через pytest-cov?

Відповідь: Командою: pytest --cov=назва_пакета --cov-branch --cov-report=term-missing.

75. Для чого --cov-report=term-missing?

Відповідь: Щоб у термінальній зведеній таблиці відображалися точні номери рядків, які не були виконані жодним тестом.

76. Для чого HTML coverage report?

Відповідь: HTML-звіт генерує зручний інтерактивний веб-сайт, де можна візуально переглядати кожен файл коду з кольоровою підсвіткою покритих (зелених) і пропущених (червоних) рядків і розгалужень.

77. Чи гарантує 100% coverage відсутність bugs?

Відповідь: Ні. 100 % coverage гарантує лише факт виконання рядків, але не перевіряє правильність бізнес-логіки та відсутність пропущених прихованих сценаріїв.

78. Чому meaningful assertions важливіші за coverage percentage?

Відповідь: Тому що лише змістовні assertions верифікують стан та гарантують виконання вимог специфікації. Тест без assert виконує код, але нічого не доводить.

79. Що таке coverage threshold?

Відповідь: Поріг покриття (наприклад, fail_under = 80) — мінімальний обов'язковий відсоток покриття, при недосягненні якого pytest завершується з ненульовим кодом помилки в CI/CD.

80. Чому tests повинні бути незалежними один від одного?

Відповідь: Щоб падіння одного тесту не викликало каскадного збою інших, щоб тести можна було запускати у довільному порядку та виконувати паралельно на кількох ядрах CPU.

9. ВИСНОВОК

У ході виконання лабораторної роботи № 6 було повноцінно та всебічно опановано сучасні стандарти, інструменти та методики автоматизованого тестування програмного забезпечення мовою Python на базі фреймворку pytest.

Основні практичні результати роботи:

1. Створено модульний проєкт системи обліку наукових експериментів (Варіант № 15) за схемою src-layout із дотриманням PEP 8 та суворої статичної типізації.

2. Розроблено вичерпний тестовий набір із 76 автоматизованих тестів (unit, integration, async, experiments), що охоплюють нормальні, граничні та виняткові сценарії функціонування системи. Усі 76 тестів проходять успішно (100 % pass rate) за 1.05 с.

3. Застосовано патерн Arrange-Act-Assert, розроблено набір фікстур у conftest.py із механізмом гарантованого очищення ресурсів yield teardown, використано параметризацію тестів із зрозумілими ідентифікаторами (ids) та граничний аналіз фізичних меж величин.

4. Опановано використання тестових двійників (Mock та Fake) із зазначенням специфікації інтерфейсу (spec), налаштуванням return_value та моделюванням аварійних сценаріїв через side_effect, перевіркою викликів методів та асинхронних корутин (AsyncMock).

5. Забезпечено ізоляцію файлових тестів за допомогою фікстури tmp_path та динамічне керування змінними середовища за допомогою monkeypatch.

6. Реалізовано повний комплекс із 7 обов'язкових методичних експериментів, які на практиці довели фундаментальні принципи тестування: відмінність виконання від верифікації, небезпеку тестів без тверджень, переваги branch coverage та детермінізму.

7. За допомогою інструменту pytest-cov досягнуто загального покриття вихідного коду (Total Coverage) на рівні 93 % (при рекомендованому мінімумі 80 %) та сформовано інтерактивний звіт у форматі HTML.